



CENTRO UNIVERSITÁRIO UNIRUY WYDEN

ARYANE SANTOS ANDRADE

RODRIGO SANTOS ARAUJO

**PLATAFORMA DE MONITORIAS ACADÊMICAS:
SISTEMA WEB PARA GESTÃO E INSCRIÇÃO EM MONITORIAS**

SALVADOR – BA

2026

ARYANE SANTOS ANDRADE

RODRIGO SANTOS ARAUJO

**PLATAFORMA DE MONITORIAS ACADÊMICAS:
SISTEMA WEB PARA GESTÃO E INSCRIÇÃO EM MONITORIAS**

Trabalho apresentado à disciplina de Engenharia de
Software da UniRuy Wyden, como requisito parcial
para avaliação da disciplina.

Orientador: Heleno Cardoso

SALVADOR – BA

2026

RESUMO

Este trabalho apresenta o desenvolvimento da Plataforma de Gestão de Monitorias Acadêmicas, uma solução web voltada à centralização e modernização do processo de divulgação e gestão de monitorias em ambiente acadêmico. O problema identificado refere-se à utilização de planilhas eletrônicas e grupos de comunicação para divulgação de monitorias, o que gera descentralização, baixa organização e dificuldade de controle das informações. A solução proposta consiste em uma aplicação web desenvolvida com o framework Django, banco de dados PostgreSQL e infraestrutura baseada em Docker. O sistema permite o gerenciamento de monitorias por monitores e a inscrição de alunos de forma centralizada, intuitiva e padronizada. O projeto aplica conceitos de engenharia de software, modelagem de sistemas, arquitetura em camadas e boas práticas de desenvolvimento web, contemplando requisitos funcionais, não funcionais, modelagem de dados e processos de implantação.

Palavras-chave: *Django. Monitorias Acadêmicas. Engenharia de Software. PostgreSQL. Sistemas Web.*

SUMÁRIO

1 PDTI – PLANO DIRETOR DE TECNOLOGIA DA INFORMAÇÃO	3
--	---

1.1 Levantamento de Custos.....	4
1.2 Estudo de Viabilidade	4
2 PROJETO TÉCNICO	5
2.1 Consultoria do Negócio	6
2.1.1 Contextualização do Problema	6
2.1.2 Descrição do Negócio	6
3 O&M – ORGANIZAÇÃO E MÉTODOS	7
3.1 Processos.....	8
3.2 Rotinas	8
3.3 Procedimentos	8
3.3.1 Procedimento de Inscrição.....	9
3.3.2 Procedimento de Gestão de Monitorias	9
4 PROJETO LÓGICO	9
4.1 Levantamento de Requisitos	10
4.2 Modelo Ambiental	10
4.2.1 Introdução.....	10
4.2.2 Objetivos e Funções.....	10
4.2.3 Diagrama de Contexto	11
4.2.4 Lista de Eventos / Requisitos Funcionais (RF)	11
4.2.5 Regras de Negócio.....	12
4.2.6 Requisitos Não Funcionais (RNF)	12
4.3 Especificação de Programas.....	13
4.4 Diagrama de Fluxo de Dados – Nível Zero (DFD Nível 0)	13
4.5 Diagrama de Fluxo de Dados por Evento.....	14
4.6 Modelagem de Dados Conceitual (MER).....	14
5 PROJETO FÍSICO.....	14
5.1 Arquitetura da Aplicação.....	15
5.2 Modelo Lógico de Dados (DER).....	15
5.3 Dicionário de Dados.....	16
5.4 Modelo Comportamental	17
5.5 Plano de Teste	18
5.6 Plano de Treinamento.....	18
5.7 Plano de Implantação	19
5.8 Layout de Telas – Protótipos e Interfaces.....	19
5.9 Telas de Diálogo e Relatórios	20
5.10 Layout de Relatórios.....	20
5.11 Especificação de Programas.....	21

6 IMPLEMENTAÇÃO	21
6.1 Desenvolvimento das Aplicações.....	22
6.2 Testes QA	22
7 IMPLANTAÇÃO	23
7.1 Instalação	24
7.2 Migração/Conversão.....	24
7.3 Treinamento	24
7.4 Validação.....	25
8 PRODUÇÃO	25
8.1 Homologação	26
8.2 Funcionamento do Sistema em Produção	26
REFERÊNCIAS	26

1 PDTI – PLANO DIRETOR DE TECNOLOGIA DA INFORMAÇÃO

1.1 Levantamento de Custos

O levantamento de custos do projeto foi realizado com base na metodologia de Unidades de Serviço Técnico (UST), conforme padrão adotado pelo mercado de tecnologia da informação e orientação do corpo docente. A precificação considera custos fixos (CF), custos variáveis (CV), impostos e margem de lucro, com referência de R\$ 160,00 por hora para profissional sênior/pleno.

Quadro 1 – Estimativa de Custos do Projeto

Item	Descrição	Qtd. Horas	Valor Unit. (R\$)	Total (R\$)
1	Levantamento de Requisitos	20h	160,00	3.200,00
2	Análise e Modelagem	30h	160,00	4.800,00
3	Desenvolvimento Backend (Django)	60h	160,00	9.600,00
4	Desenvolvimento Frontend (Templates)	30h	160,00	4.800,00
5	Configuração de Infraestrutura (Docker/Nginx)	20h	160,00	3.200,00
6	Testes e Validação	15h	160,00	2.400,00
7	Documentação e Implantação	15h	160,00	2.400,00
	TOTAL	190h	—	30.400,00

1.2 Estudo de Viabilidade

O estudo de viabilidade avaliou os aspectos técnico, operacional, econômico e de prazo do projeto.

Do ponto de vista técnico, a solução utiliza tecnologias consolidadas e amplamente adotadas no mercado: Django (Python), PostgreSQL, Docker e Nginx. A equipe de desenvolvimento possui familiaridade com o ecossistema, tornando o projeto viável nessa dimensão.

Operacionalmente, o sistema substitui um processo manual baseado em planilhas Excel compartilhadas por grupos de WhatsApp, eliminando redundâncias,

descentralização de informações e dificuldades de atualização. A plataforma proposta centraliza todas as informações em um único ambiente digital, acessível via navegador.

Do ponto de vista econômico, o custo estimado de R\$ 30.400,00 é compatível com projetos de pequeno porte voltados para o setor educacional, com potencial de extensão para outros cursos e unidades da instituição. A viabilidade econômica é reforçada pela redução de retrabalho administrativo e melhoria na qualidade do serviço prestado aos alunos.

O cronograma estimado é de aproximadamente quatro meses, divididos entre as etapas de análise, desenvolvimento, testes e implantação, o que é adequado para o escopo definido.

2 PROJETO TÉCNICO

2.1 Consultoria do Negócio

2.1.1 Contextualização do Problema

O projeto foi desenvolvido com o objetivo de solucionar uma dificuldade recorrente no contexto acadêmico da Universidade UniRuy Wyden, relacionada à divulgação de monitorias por meio de planilhas Excel compartilhadas em grupos de WhatsApp. Essa abordagem resulta em descentralização das informações, dificuldades de atualização e baixa organização dos dados, comprometendo a eficiência do processo.

Diante desse cenário, a plataforma de gestão de monitorias propõe a centralização e modernização desse fluxo de informação, oferecendo uma solução estruturada, acessível e eficiente. O sistema permite a autenticação de monitores, o gerenciamento completo das monitorias, além do controle de inscrições e de vagas disponíveis.

Com isso, busca-se promover maior organização, transparência e facilidade de acesso às informações, beneficiando tanto os alunos quanto os monitores envolvidos no processo acadêmico.

2.1.2 Descrição do Negócio

O sistema implementa uma plataforma de gerenciamento de monitorias acadêmicas com duas classes de usuário distintas:

- Aluno: pode consultar as monitorias disponíveis e realizar inscrições.
- Monitor: deve cadastrar conta, autenticar-se na plataforma e gerenciar suas próprias monitorias.

A consultoria do negócio identificada abrange as seguintes funcionalidades principais:

- Publicação de monitorias com categoria, benefício, data e número de vagas disponíveis.
- Controle de inscrições com limitação automática de vagas.
- Painel de gestão para o monitor com operações de edição, exclusão e listagem de inscritos.

3 O&M – ORGANIZAÇÃO E MÉTODOS

3.1 Processos

O sistema opera mediante os seguintes processos principais, implementados no módulo views.py das aplicações Django:

- Cadastro de usuário: processado pela view de registro (register_view).
- Autenticação: realizada pela view de login (login_view).
- Listagem pública de monitorias: executada pela view de listagem (MonitoriaListView).
- Criação, edição e exclusão: disponíveis exclusivamente para usuários autenticados.
- Inscrição de aluno em monitoria: processada pelo formulário na view MonitorialnscricaoView.

3.2 Rotinas

As rotinas identificadas no sistema são:

- register_view: cria usuário com o formulário RegisterForm.
- login_view: autentica usuário com o formulário LoginForm.
- logout_view: limpa a sessão e redireciona para a lista pública de monitorias.
- MonitoriaListView.get_queryset(): realiza pesquisa textual e filtra por categoria.
- NovaMonitoriaCreateView.form_valid(): associa o campo monitor ao request.user autenticado.
- MonitoriaUpdateView / MonitoriaDeleteView: operam exclusivamente sobre monitorias cujo monitor corresponde ao usuário autenticado.
- MonitorialnscricaoView.post(): valida a lotação da monitoria e impede inscrição do próprio usuário (monitor).

3.3 Procedimentos

3.3.1 Procedimento de Inscrição

O fluxo de inscrição segue as etapas descritas a seguir:

1. O usuário acessa o endereço /monitorias/<pk>/inscricao/.
2. O sistema verifica se o usuário não é o monitor da monitoria.
3. O sistema avalia se a monitoria está com vagas disponíveis.
4. O formulário valida os dados e registra o objeto Inscricao no banco de dados.

3.3.2 Procedimento de Gestão de Monitorias

1. O monitor acessa o painel em /painel/.
2. A partir do painel, o monitor pode criar, editar e excluir suas monitorias.
3. O monitor pode acessar /monitorias/<pk>/inscritos/ para visualizar a lista de inscritos em cada monitoria.

4 PROJETO LÓGICO

4.1 Levantamento de Requisitos

O levantamento de requisitos foi realizado a partir da análise do problema identificado na instituição. Os requisitos levantados são apresentados a seguir.

Quadro 2 – Requisitos do Sistema

Código	Descrição
R1	Permitir navegação pública por monitorias disponíveis.
R2	Permitir cadastro de monitor via formulário de usuário.
R3	Permitir login e logout de monitor.
R4	Permitir criação de monitoria com categoria, benefício, data e vagas.
R5	Permitir edição e exclusão de monitoria pelo monitor.
R6	Permitir inscrição em monitoria com nome, e-mail e telefone.
R7	Controlar vagas e bloquear inscrição quando a monitoria estiver lotada.
R8	Permitir visualização de inscritos apenas pelo monitor da monitoria.

4.2 Modelo Ambiental

4.2.1 Introdução

O modelo ambiental descreve o contexto no qual o sistema está inserido, identificando os agentes externos que interagem com ele e os principais objetivos e funções a serem atendidos. A Plataforma de Monitorias Acadêmicas foi concebida para operar no ambiente da Universidade UniRuy Wyden, com acesso via navegador web, sem necessidade de instalação de software pelos usuários finais.

4.2.2 Objetivos e Funções

O objetivo principal do sistema é gerenciar monitorias acadêmicas e inscrições de alunos de forma centralizada, substituindo o processo informal baseado em planilhas compartilhadas. As funções principais são:

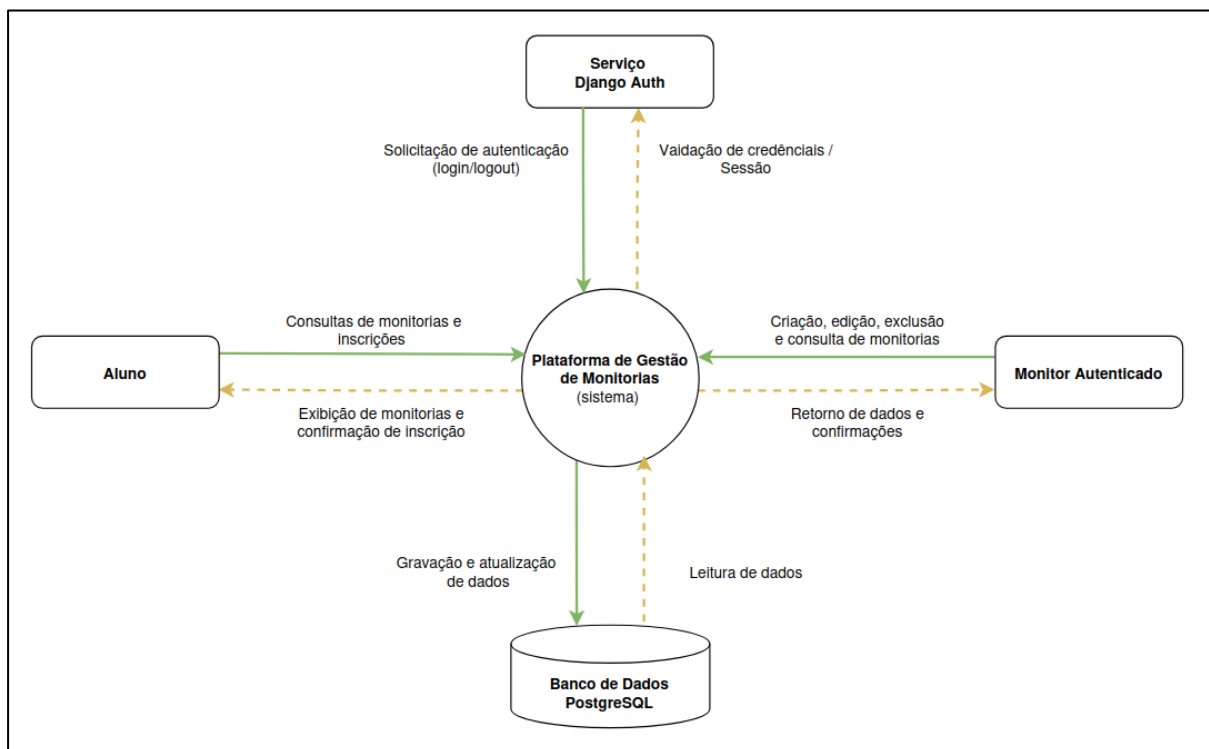
- Consulta pública de monitorias disponíveis.

- Registro e autenticação de monitores.
- Criação, edição e exclusão de monitorias.
- Inscrição de usuários em monitorias.
- Visualização de inscritos pelo monitor responsável.

4.2.3 Diagrama de Contexto

O Diagrama de Contexto apresenta o sistema como uma caixa-preta, evidenciando suas interações com os agentes externos. Os agentes externos são: Usuário/Aluno, Monitor Autenticado, Banco de Dados PostgreSQL e Serviço Django Auth.

Figura 1 – Diagrama de Contexto



4.2.4 Lista de Eventos / Requisitos Funcionais (RF)

Os requisitos funcionais descrevem as funcionalidades que o sistema deve oferecer. São apresentados na tabela a seguir, acompanhados das regras de negócio associadas.

Quadro 3 – Requisitos Funcionais

Código	Descrição	Verbo
RF01	Permitir listagem de monitorias.	Informar
RF02	Permitir busca e filtro de monitorias por texto e categoria.	Informar
RF03	Permitir cadastro de usuário monitor.	Informar
RF04	Permitir autenticação e manutenção de sessão.	Informar
RF05	Permitir criação de monitoria.	Gerar
RF06	Permitir edição de monitoria pelo monitor.	Gerar
RF07	Permitir exclusão de monitoria pelo monitor.	Gerar
RF08	Permitir inscrição de aluno em monitoria.	Gerar
RF09	Permitir listagem de inscritos por monitor.	Solicitar

Quadro 4 – Lista de Eventos do Sistema

Evento	Origem	Destino	Efeito
Login	Usuário	login_view	Cria sessão autenticada
Logout	Usuário	logout_view	Encerra sessão
Cadastro de monitoria	Monitor	NovaMonitoriaCreateView	Insere objeto Monitoria
Edição de monitoria	Monitor	MonitoriaUpdateView	Atualiza objeto Monitoria
Exclusão de monitoria	Monitor	MonitoriaDeleteView	Remove objeto Monitoria
Inscrição em monitoria	Aluno	MonitorialnscricaoView	Insere objeto Inscricao
Visualizar inscritos	Monitor	MonitorialnscritoListView	Lista objetos Inscricao

4.2.5 Regras de Negócio

- RN01: Apenas o monitor criador pode editar ou excluir uma monitoria.
- RN02: O monitor não pode se inscrever na própria monitoria.
- RN03: Quando o número de inscrições atingir o limite de vagas, novas inscrições devem ser bloqueadas.

- RN04: O e-mail do inscrito deve ser único por monitoria, evitando duplicidade de inscrição.
- RN05: Para criar, editar ou excluir monitorias, o usuário deve estar autenticado.

4.2.6 Requisitos Não Funcionais (RNF)

Quadro 5 – Requisitos Não Funcionais

Código	Categoria	Descrição
RNF01	Tecnologia	Aplicação web construída com Django 6.0.5 (Python).
RNF02	Banco de Dados	Banco de dados relacional PostgreSQL 15.
RNF03	Infraestrutura	Containerização via Docker e Docker Compose.
RNF04	Interface	Interface responsiva desenvolvida com Bootstrap 5.
RNF05	Segurança	Proteção CSRF nativa do Django e controle de sessões.
RNF06	Servidor	Serviço web exposto via Gunicorn com proxy Nginx.
RNF07	Configuração	Variáveis de ambiente para configuração sensível.
RNF08	Armazenamento	Arquivos estáticos armazenados em volume Docker.

4.3 Especificação de Programas

As classes e funções principais identificadas no sistema são:

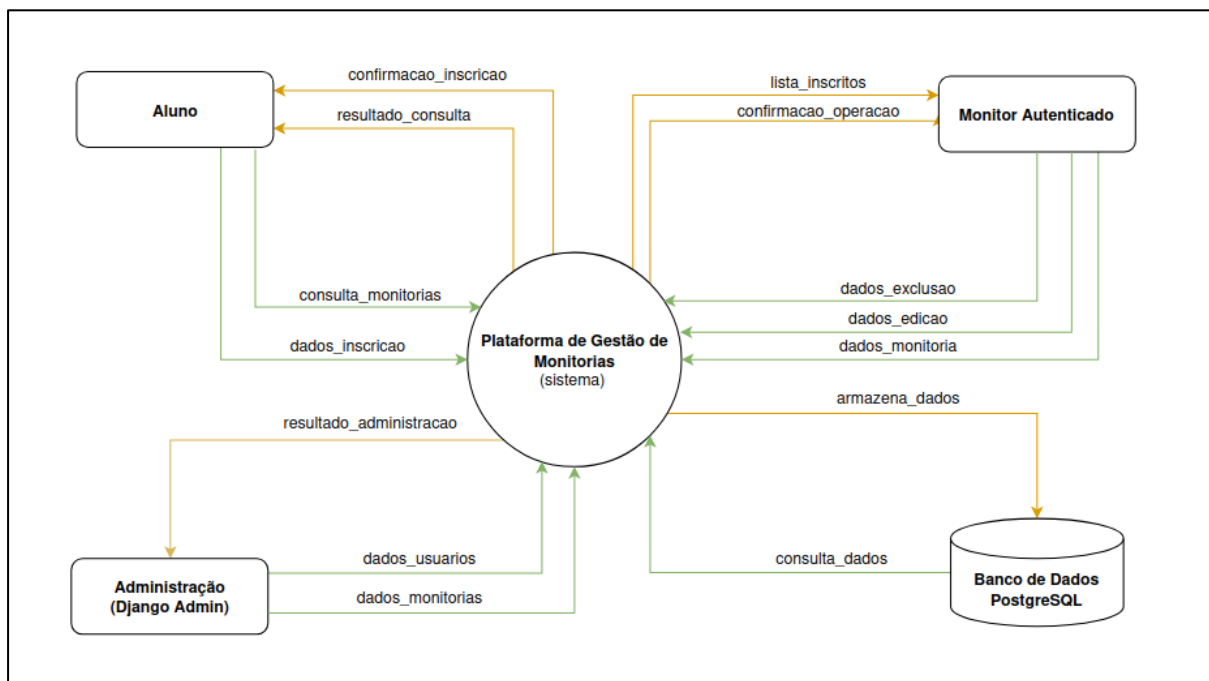
- `contas.views.register_view`: responsável pelo cadastro de novos monitores.
- `contas.views.login_view`: responsável pela autenticação do usuário.
- `contas.views.logout_view`: responsável pelo encerramento da sessão.
- `monitorias.views.MonitoriaListView`: exibe a listagem pública de monitorias com suporte a busca e filtro.
- `monitorias.views.NovaMonitoriaCreateView`: permite a criação de novas monitorias por monitores autenticados.
- `monitorias.views.MonitoriaUpdateView`: permite a edição de monitorias pelo monitor.

- `monitorias.views.MonitoriaDeleteView`: permite a exclusão de monitorias pelo monitor.
- `monitorias.views.MonitorialInscricaoView`: processa o formulário de inscrição de alunos.
- `monitorias.views.MonitorialInscritoListView`: lista os inscritos de uma monitoria para o monitor.
- `monitorias.views.painel_view`: exibe o painel de gestão do monitor.

4.4 Diagrama de Fluxo de Dados – Nível Zero (DFD Nível 0)

O DFD Nível 0 representa os principais processos do sistema e seus fluxos de dados. O sistema é composto pelos seguintes processos principais: Autenticação, Listagem de Monitorias, CRUD de Monitoria e Inscrição em Monitoria. Todos os processos se comunicam com o repositório de dados (Banco de Dados PostgreSQL).

Figura 2 – Diagrama de Fluxo de Dados Nível Zero



4.5 Diagrama de Fluxo de Dados por Evento

O DFD por Evento detalha o fluxo de dados para cada evento principal do sistema, conforme a Figura 3 e Figura 4.

Figura 3 – Inscrição em monitoria

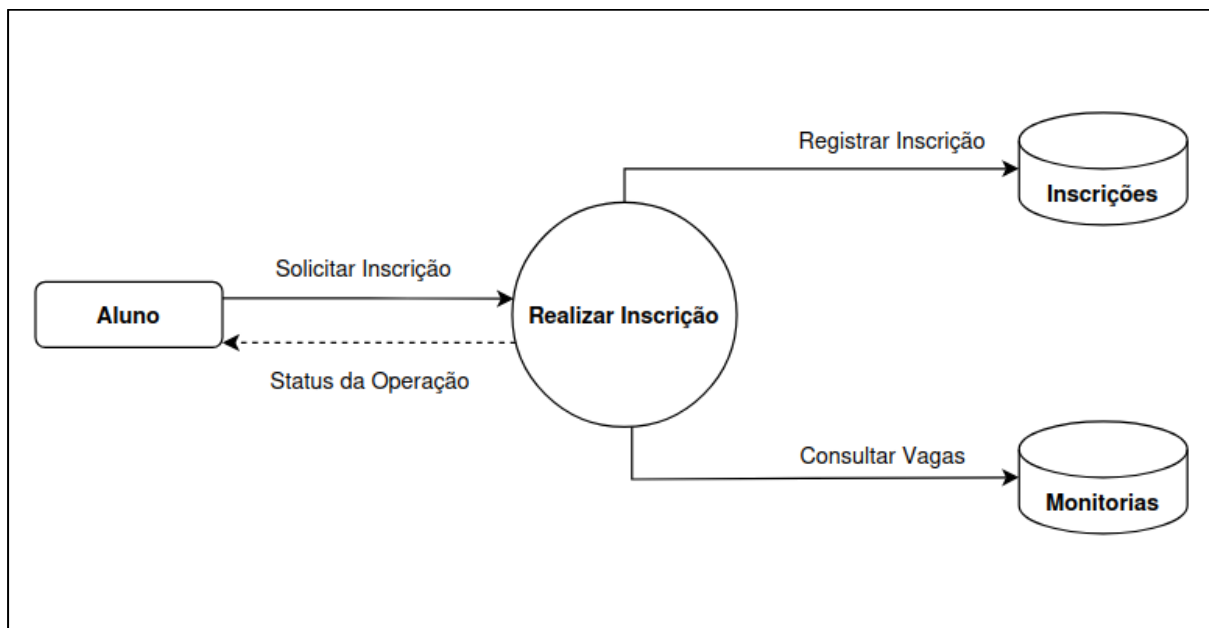
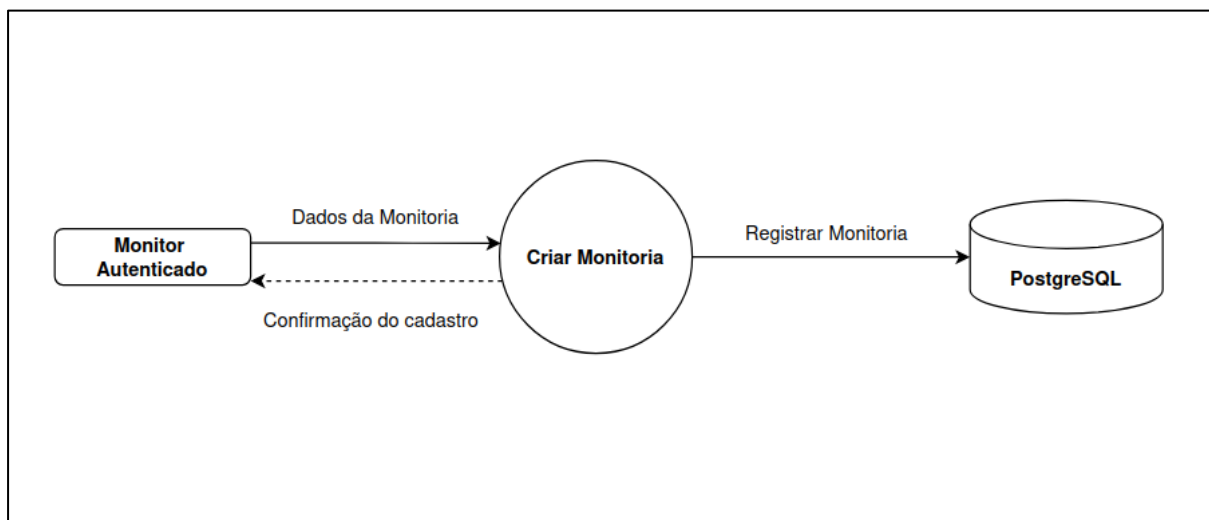


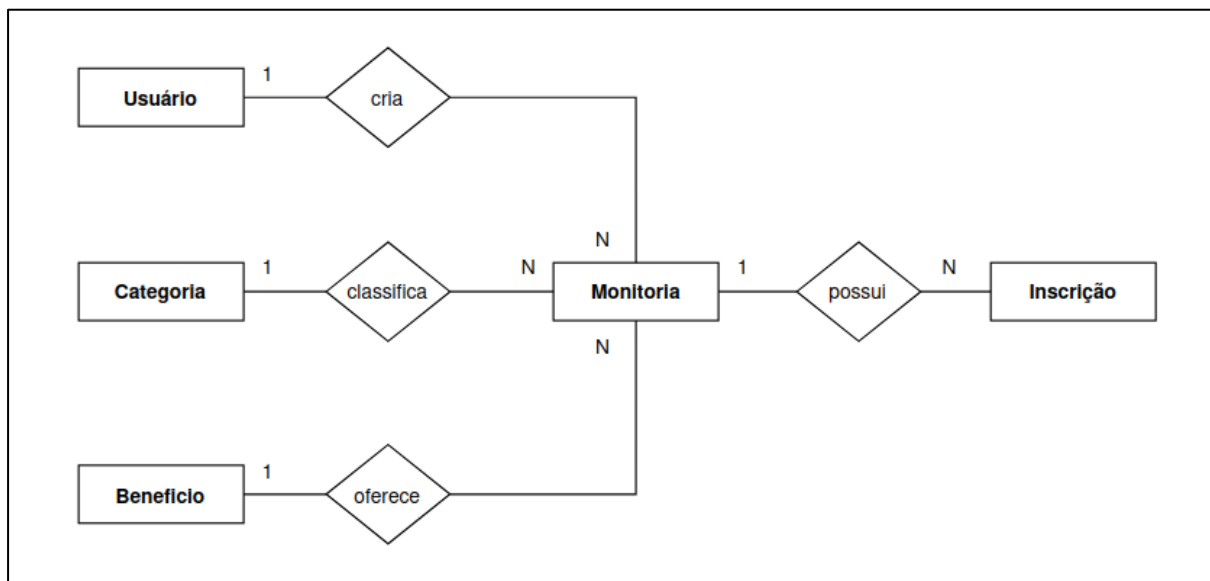
Figura 4 – Criação de monitoria



4.6 Modelagem de Dados Conceitual (MER)

O Modelo Entidade-Relacionamento Conceitual (MER) é independente do banco de dados utilizado. As entidades e relacionamentos identificados no sistema são apresentados a seguir, utilizando a notação padrão (Retângulo = Entidade; Losango = Relacionamento).

Figura 4 – Entidades e Atributos (MER Conceitual)



5 PROJETO FÍSICO

5.1 Arquitetura da Aplicação

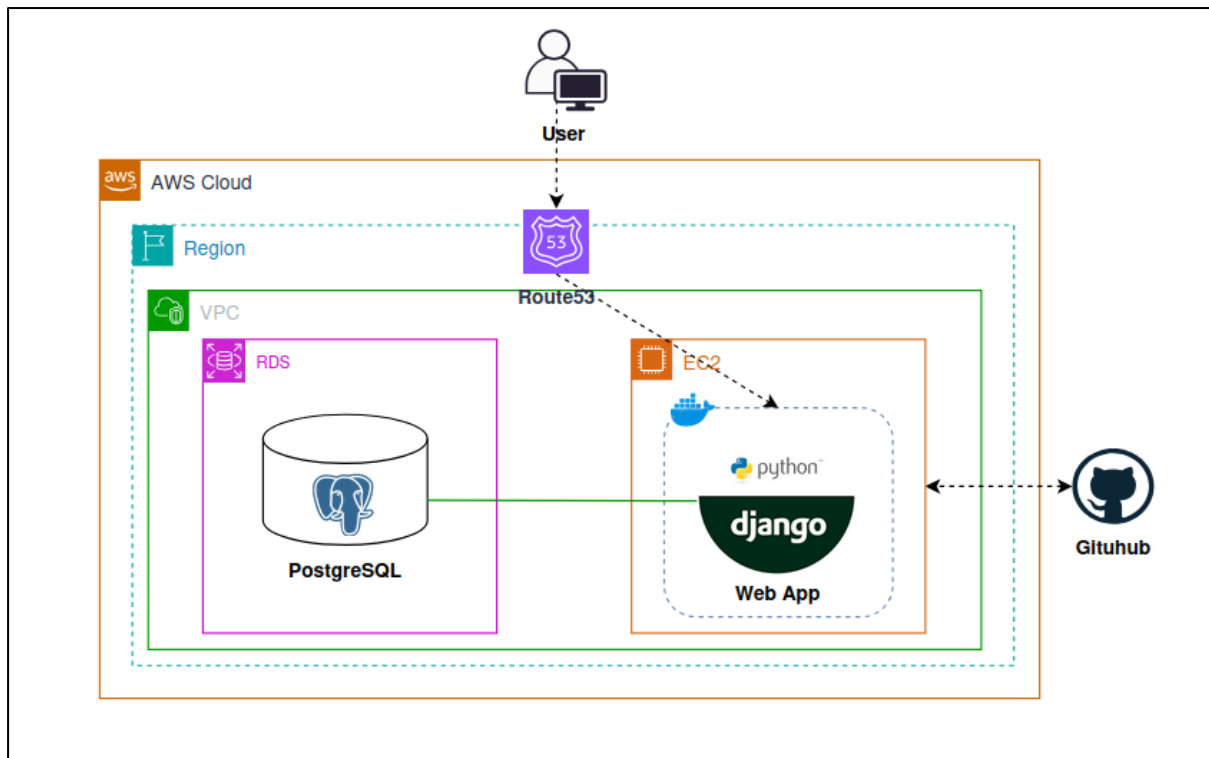
O sistema adota uma arquitetura em três camadas, seguindo o padrão MVT (Model-View-Template) do framework Django:

- Camada de Apresentação: templates Django responsivos com Bootstrap 5, responsáveis pela interface com o usuário.
- Camada de Negócio: views e forms Django, responsáveis pela lógica de negócio, validações e regras de acesso.
- Camada de Persistência: models Django mapeados para tabelas PostgreSQL via ORM.

A infraestrutura de implantação é baseada em contêineres Docker, com os seguintes componentes:

- Contêiner db: PostgreSQL 15 para persistência de dados.
- Contêiner web: Django/Gunicorn para execução da aplicação.
- Contêiner nginx: servidor web para proxy reverso, HTTPS e servimento de arquivos estáticos.

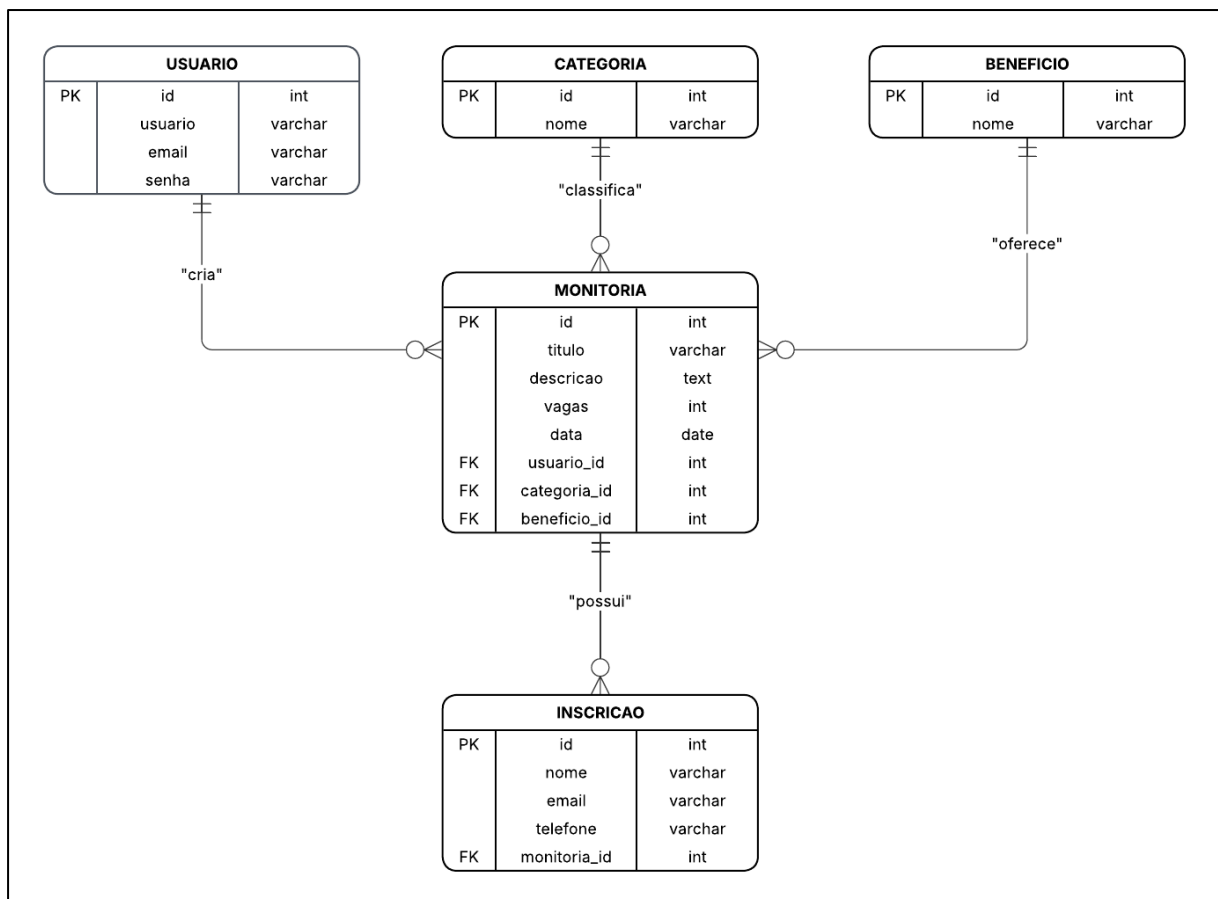
Figura 5 – Arquitetura de Implantação da Aplicação na AWS



5.2 Modelo Lógico de Dados (DER)

O Diagrama Entidade-Relacionamento (DER) representa o modelo lógico de dados dependente da tecnologia PostgreSQL. As chaves estrangeiras e cardinalidades são representadas a seguir.

Figura 6 – Relacionamentos do DER



5.3 Dicionário de Dados

O dicionário de dados descreve os atributos de cada entidade do sistema, incluindo tipo, restrições e descrição.

Quadro 6 – Dicionário de Dados: Entidade Categoria

Atributo	Tipo	Restrição	Descrição
id	AutoField (INT)	PK, NOT NULL	Identificador único da categoria
nome	CharField(100)	NOT NULL	Nome da categoria de monitoria

Quadro 7 – Dicionário de Dados: Entidade Beneficio

Atributo	Tipo	Restrição	Descrição
id	AutoField (INT)	PK, NOT NULL	Identificador único do beneficio

nome	CharField(100)	NOT NULL	Nome do benefício oferecido pela monitoria
------	----------------	----------	--

Quadro 8 – Dicionário de Dados: Entidade Monitoria

Atributo	Tipo	Restrição	Descrição
id	AutoField (INT)	PK, NOT NULL	Identificador único da monitoria
titulo	CharField(200)	NOT NULL	Título descritivo da monitoria
descricao	TextField	NOT NULL	Descrição detalhada da monitoria
data	DateField	NULL (opcional)	Data de início da monitoria
categoria	ForeignKey	PROTECT, NOT NULL	Categoria associada à monitoria
beneficio	ForeignKey	PROTECT, NOT NULL	Benefício associado à monitoria
vagas	PositiveIntegerField	DEFAULT=1	Total de vagas disponíveis
owner (monitor)	ForeignKey (User)	CASCADE, NOT NULL	Usuário monitor responsável

Quadro 9 – Dicionário de Dados: Entidade Inscricao

Atributo	Tipo	Restrição	Descrição
id	AutoField (INT)	PK, NOT NULL	Identificador único da inscrição
monitoria	ForeignKey	CASCADE, NOT NULL	Monitoria na qual o aluno se inscreveu
nome	CharField(200)	NOT NULL	Nome completo do inscrito
email	EmailField	NOT NULL, UNIQUE por monitoria	E-mail do inscrito
telefone	CharField(20)	NOT NULL	Telefone de contato do inscrito

5.4 Modelo Comportamental

O modelo comportamental descreve o comportamento dinâmico do sistema mediante as principais regras de autorização e validação implementadas:

- Autorização por usuário: as operações de edição, exclusão e visualização de inscritos são restritas ao monitor da monitoria, verificado em cada view.

- Validação de inscrição: o método `InscricaoModelForm.clean_email()` verifica a unicidade do e-mail por monitoria, evitando inscrições duplicadas.
- Controle de lotação: o método `Monitoria.monitoria_lotada()` compara o total de inscrições com o número de vagas disponíveis, bloqueando novas inscrições quando atingido o limite.

5.5 Plano de Teste

O plano de testes define os cenários a serem validados para garantia da qualidade do sistema. Os testes foram realizados para validar as funcionalidades do sistema.

Quadro 10 – Casos de Teste Planejados

ID	Cenário de Teste	Resultado Esperado
CT01	Cadastro de novo monitor com dados válidos	Usuário criado e redirecionado para login
CT02	Login com credenciais válidas	Sessão iniciada e redirecionamento ao painel
CT03	Login com credenciais inválidas	Mensagem de erro exibida
CT04	Criação de monitoria autenticado	Monitoria inserida no banco de dados
CT05	Tentativa de editar monitoria de outro monitor	Acesso negado (403 ou redirecionamento)
CT06	Inscrição em monitoria com vagas disponíveis	Inscrição registrada com sucesso
CT07	Inscrição em monitoria lotada	Mensagem de lotação exibida; inscrição bloqueada
CT08	Monitor tenta se inscrever na própria monitoria	Inscrição bloqueada pela regra de negócio
CT09	Inscrição duplicada com mesmo e-mail	Validação impede duplicidade

5.6 Plano de Treinamento

O plano de treinamento contempla os seguintes perfis de usuário e suas respectivas necessidades:

- Monitores: treinamento de aproximadamente 2 horas, abordando cadastro de conta, criação e gerenciamento de monitorias e consulta de inscritos.
- Alunos: orientação de 30 minutos, cobrindo navegação pela plataforma, busca e filtragem de monitorias e preenchimento do formulário de inscrição.
- Administradores: treinamento técnico de 4 horas, incluindo operação do ambiente Docker, execução de migrações, monitoramento de logs e backup do banco de dados.

5.7 Plano de Implantação

A implantação do sistema segue as etapas descritas abaixo, baseadas na configuração Docker do projeto:

Quadro 11 – Etapas do Plano de Implantação

Etapa	Descrição	Comando / Ação
1. Preparação do ambiente	Instalar Docker e Docker Compose no servidor de produção	Instalação via apt/yum
2. Configuração de variáveis	Definir variáveis de ambiente (SECRET_KEY, DB_PASSWORD, ALLOWED_HOSTS)	Arquivo .env
3. Build da imagem	Construir a imagem Docker com as dependências do requirements.txt	docker-compose -f docker-compose.prod.yml build
4. Migração do banco	Executar as migrações Django para criar as tabelas	python manage.py migrate
5. Coleta de estáticos	Coletar arquivos estáticos para o volume Docker	python manage.py collectstatic
6. Inicialização	Subir os contêineres em modo produção	docker-compose -f docker-compose.prod.yml up -d
7. Validação	Verificar funcionamento dos serviços db, web e nginx	docker ps / testes manuais

5.8 Layout de Telas – Protótipos e Interfaces

As principais telas do sistema são descritas a seguir, com seus respectivos endpoints e funcionalidades:

Quadro 12 – Descrição das Telas do Sistema

Tela	Endpoint	Acesso	Descrição
Listagem de Monitorias	/monitorias/	Público	Exibe todas as monitorias com campos de busca e filtro por categoria.
Inscrição em Monitoria	/monitorias/<pk>/inscricao/	Público	Formulário com campos nome, e-mail e telefone para inscrição.
Cadastro de Monitor	/registro/	Público	Formulário de criação de conta de monitor.
Login	/login/	Público	Formulário de autenticação com username e senha.
Painel do Monitor	/painel/	Autenticado	Lista as monitorias do monitor com ações de edição e exclusão.
Criar Monitoria	/criar_monitoria/	Autenticado	Formulário completo para criação de nova monitoria.
Editar Monitoria	/painel/<pk>/editar_monitoria	Monitor	Formulário pré-preenchido para edição da monitoria.
Excluir Monitoria	/painel/<pk>/excluir_monitoria	Monitor	Confirmação de exclusão da monitoria.
Lista de Inscritos	/monitorias/<pk>/inscritos/	Monitor	Tabela com nome, e-mail e telefone dos inscritos.

5.9 Telas de Diálogo e Relatórios

O sistema não possui módulo de relatórios gerados em formato de arquivo (PDF, Excel ou similar). As informações são apresentadas diretamente nas telas de listagem, que constituem os relatórios operacionais do sistema:

- Listagem de Monitorias: relatório público com título, categoria, benefício, data e vagas disponíveis.
- Listagem de Inscritos: relatório restrito ao monitor, com nome, e-mail e telefone dos inscritos em cada monitoria.

5.10 Layout de Relatórios

Os relatórios são exibidos em formato de tabela responsiva (Bootstrap 5) diretamente no navegador, sem geração de arquivos externos. A estrutura das tabelas segue o padrão de cabeçalho fixo com linhas alternadas para facilitar a leitura. A implementação de exportação para PDF ou planilha constitui uma melhoria prevista para versões futuras do sistema.

5.11 Especificação de Programas

Quadro 13 – Endpoints HTTP do Sistema

Rota	Método	View	Descrição
/	GET	Redirect	Redireciona para a listagem de monitorias
/monitorias/	GET	MonitoriaListView	Listagem pública de monitorias com busca e filtro
/monitorias/<pk>/inscricao/	GET/POST	MonitoriaInscricaoView	Formulário de inscrição em monitoria
/monitorias/<pk>/inscritos/	GET	MonitoriaInscritoListView	Lista inscritos (restrito ao monitor)
/criar_monitoria/	GET/POST	NovaMonitoriaCreateView	Criação de monitoria (autenticado)
/painel/<pk>/editar_monitoria	GET/POST	MonitoriaUpdateView	Edição de monitoria (monitor)
/painel/<pk>/excluir_monitoria	GET/POST	MonitoriaDeleteView	Exclusão de monitoria (monitor)
/registro/	GET/POST	register_view	Cadastro de novo monitor
/login/	GET/POST	login_view	Autenticação de usuário
/logout/	GET	logout_view	Encerramento de sessão
/painel/	GET	painel_view	Painel de gestão do monitor

6 IMPLEMENTAÇÃO

6.1 Desenvolvimento das Aplicações

O desenvolvimento foi realizado utilizando o framework Django (Python), seguindo o padrão de projeto MVT (Model-View-Template). O projeto é organizado em dois aplicativos principais:

- **contas:** responsável pelo gerenciamento de usuários, incluindo cadastro, login e logout, utilizando `UserCreationForm` e `AuthenticationForm` do Django.
- **monitorias:** responsável por toda a lógica de negócio relacionada às monitorias e inscrições, incluindo models, views baseadas em classes (CBV) e `ModelForms`.

As principais tecnologias e bibliotecas utilizadas são:

- Django 6.0.5: framework web principal.
- PostgreSQL 15: banco de dados relacional.
- Bootstrap 5: framework CSS para interface responsiva.
- Gunicorn: servidor WSGI para produção.
- Nginx: servidor web/proxy reverso.
- Docker e Docker Compose: containerização e orquestração.

6.2 Testes QA

Na versão atual do sistema, os arquivos `tests.py` das aplicações monitorias e contas estão estruturados, porém sem casos de teste implementados. A ausência de testes automatizados representa um ponto de atenção e melhoria para versões futuras.

Recomenda-se a implementação de testes unitários para os models (especialmente os métodos `monitoria_lotada` e `clean_email`), testes de integração

para as views e testes de aceitação para os fluxos críticos (inscrição e gestão de monitorias), utilizando o módulo unittest do Django.

7 IMPLANTAÇÃO

7.1 Instalação

O ambiente é provisionado inteiramente por meio de Docker, garantindo portabilidade e padronização entre os ambientes de desenvolvimento e produção. Os comandos de instalação são:

Ambiente de desenvolvimento:

```
docker-compose up --build
```

Ambiente de produção:

```
docker-compose -f docker-compose.prod.yml up --build
```

O Dockerfile define um ambiente Python 3.12 com todas as dependências listadas no arquivo requirements.txt, incluindo Django, psycopg2 (driver PostgreSQL) e Gunicorn.

7.2 Migração/Conversão

As migrações de banco de dados estão presentes no diretório migrations das aplicações. A execução das migrações é realizada com o comando padrão do Django:

```
python manage.py migrate
```

Não há necessidade de migração de dados legados, uma vez que o sistema substitui um processo manual (planilhas Excel) sem estrutura de banco de dados prévia.

7.3 Treinamento

O treinamento dos usuários segue o plano descrito no item 5.6 deste documento. Recomenda-se a elaboração de um manual de usuário simplificado para monitores e alunos, abordando as funcionalidades principais por meio de capturas de tela anotadas.

7.4 Validação

A validação do sistema deve ser realizada por meio de testes de aceitação junto aos usuários finais (monitores e alunos da UniRuy Wyden), verificando se os requisitos funcionais e não funcionais levantados foram atendidos. Recomenda-se a realização de um piloto com um grupo restrito de monitores antes da implantação em produção completa.

8 PRODUÇÃO

8.1 Homologação

A homologação do sistema deve ser conduzida em ambiente de staging, replicando as condições de produção. O processo de homologação deve validar todos os casos de teste descritos no Plano de Testes (item 5.5), com participação dos stakeholders (coordenação acadêmica, monitores e representantes dos alunos).

A homologação deve contemplar também a verificação de desempenho sob carga típica de uso, segurança (proteção CSRF, autenticação), responsividade da interface em diferentes dispositivos e integridade dos dados no banco de dados PostgreSQL.

8.2 Funcionamento do Sistema em Produção

O sistema em ambiente de produção opera com a seguinte arquitetura de serviços Docker:

- nginx: responsável por servir os arquivos estáticos (imagens, CSS, JavaScript) e realizar o proxy reverso das requisições HTTP/HTTPS para o serviço web.
- web: executa a aplicação Django via Gunicorn, processando as requisições da aplicação.
- db: instância PostgreSQL responsável pela persistência de todos os dados da plataforma.
- static_volume: volume Docker que armazena os arquivos estáticos coletados via `manage.py collectstatic`.

A configuração do Nginx (`default.conf`) define o proxy reverso, os alias para arquivos estáticos e o suporte a HTTPS via TLS, garantindo segurança na comunicação entre cliente e servidor.

REFERÊNCIAS

DJANGO SOFTWARE FOUNDATION. Django documentation. Versão 6.0. Disponível em: <<https://docs.djangoproject.com/>>. Acesso em: mai. 2026.

DOCKER INC. Docker documentation. Disponível em: <<https://docs.docker.com/>>. Acesso em: mai. 2026.

NGINX INC. NGINX documentation. Disponível em: <<https://nginx.org/en/docs/>>. Acesso em: mai. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL 15 documentation. Disponível em: <<https://www.postgresql.org/docs/15/>>. Acesso em: mai. 2026.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 6023: Informação e documentação – Referências – Elaboração. Rio de Janeiro: ABNT, 2018.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 6024: Informação e documentação – Numeração progressiva das seções de um documento – Apresentação. Rio de Janeiro: ABNT, 2012.

PRESSMAN, Roger S. Engenharia de software: uma abordagem profissional. 7. ed. Porto Alegre: AMGH, 2011.